



IUBAT-International University of Business Agriculture and Technology

LAB PROJECT

Project Title: Emergency Potable Water Allocation for a Flood-

Affected Municipality Course CSC 1384 – Structured

Programming LAB

Submitted By

Student 1: Mohammed Khaled Bin Ibrahim

Student ID: 26103014

Student 2: MD. Jobayer Islam

Student ID: 26103004

Section : E

Course Instructor

Abdullah Mohammad Sakib

Lecturer

Department of CSE

Date Of Submission:10/09/2026

Introduction

Water is one of the most important resources people need during an emergency. During a flood, the normal water supply system may not work properly. Roads can become difficult to use, pipelines may be damaged, electricity may be cut off, and nearby water sources can become contaminated. Because of this, emergency authorities may have only a limited amount of safe drinking water to distribute. This water needs to be shared among different service zones, such as hospitals, emergency shelters, residential areas, and emergency services.

In this assignment, a temporary water distribution centre has 13,500 litres of potable water available for six affected service zones. However, the total amount requested by the zones is more than the available supply. For this reason, simply dividing the water equally or distributing it according to the original order would not be the best solution. Several factors need to be considered when deciding which zones should receive water first. These include the amount of water requested, the minimum requirement of each zone, the type of service zone, how long the zone has been waiting, possible distribution losses, the amount of water available, and any remaining unmet demand.

To solve this problem, the project uses a C program to analyse the information for all six zones and determine how the available water should be distributed. Arrays are used to store the data for each zone. The program calculates a priority based on the previous waiting cycles and then uses Bubble Sort to arrange the zones according to their priority. After sorting, the water is distributed in two stages. First, each zone receives its minimum required allocation as far as the available water allows. The remaining water is then distributed based on the priority order. This approach provides a more organised and practical way to distribute limited water during an emergency.

1. Problem Analysis

2. A severe flood has disrupted the normal drinking-water supply in a municipality. Several problems, including contaminated water sources, damaged pipelines, power outages, and difficulties with transportation, have made it harder to provide safe drinking water. At the temporary distribution centre, only 13,500 litres of potable water are available, while the six affected service zones have requested a total of 20,400 litres for the current distribution cycle. This means there is already a shortage of 6,900 litres, even before considering any water that may be lost during distribution.
3. Because the available water is not enough to meet all the requests, the programme needs to decide how the 13,500 litres should be divided among the six zones. The allocation should not be based simply on equal distribution or the order in which the zones were listed. Instead, the programme should consider the needs and priorities of each zone so that the available water can be distributed in a fair, organised, and justifiable way.

1.1 The supplied dataset

The six service zones and their recorded attributes are summarised below. These values are the input to the programme and are not fixed inside the allocation logic — only inside the data-initialisation function, so that they can be changed for testing without altering how the allocation decision is made.

Zone ID	Service Zone	Type	Requested (L)	Minimum (L)	Loss (%)	Waiting Cycles
Z01	Municipal Hospital	Hospital	4,000	3,000	2	0
Z02	Central Flood Shelter	Emergency Shelter	3,500	2,500	4	1
Z03	Ward 3 Residential Area	Residential	4,500	2,000	8	2
Z04	Ward 5 Residential Area	Residential	3,800	1,800	12	3
Z05	School Emergency Shelter	Emergency Shelter	2,600	1,600	5	1
Z06	Fire and Emergency Service	Emergency Service	2,000	1,500	1	0

Totals: 20,400 L requested; 12,400 L combined minimum requirement. Because 12,400 L is still below the 13,500 L available, every zone's minimum can, in principle, be met in the original scenario — but only if the water actually released is large enough to absorb each zone's distribution loss on the way there. This distinction between water released and water received is central to the whole problem and is discussed in Section 1.4.

1.2 Zone-by-zone analysis

The six service zones have different levels of importance and different water needs. Because the available water is limited, each zone cannot simply be treated in the same way.

Municipal Hospital (Z01) has the highest life-safety priority in the dataset. Since the hospital is treating people affected by the flood, it needs a reliable minimum supply of water. It has not waited for any previous distribution cycles, so its priority comes mainly from the critical role it plays rather than from waiting time.

Fire and Emergency Service (Z06) has the smallest water request, at 2,000 litres, and the lowest distribution loss, at only 1%. This makes it relatively efficient to serve while also being important for emergency response. Like the hospital, it has zero previous waiting cycles.

Central Flood Shelter (Z02) and School Emergency Shelter (Z05) are both responsible for housing people who have been displaced by the flood. They therefore need a reasonable supply of water for immediate needs. Both zones have waited for one previous cycle, which gives them some additional priority based on fairness. Their distribution losses are 4% for Z02 and 5% for Z05.

Ward 3 Residential Area (Z03) and Ward 5 Residential Area (Z04) are residential zones. They have the largest water requests, at 4,500 litres and 3,800 litres, respectively. However, they also have relatively low minimum requirements compared with what they requested and the

highest distribution losses, at 8% for Z03 and 12% for Z04. At the same time, they have waited the longest, with Z03 waiting for two cycles and Z04 for three cycles.

These two residential zones show an important challenge in the allocation process. Their long waiting times give them a strong claim based on fairness, but their lower criticality and higher water losses may cause them to receive less priority than hospitals, emergency services, and shelters.

1.3 Why simple allocation rules are inadequate

Three naive strategies were considered and rejected before designing the strategy in Section 2, each illustrated with the numbers from this dataset.

The six service zones are different from each other in terms of their importance and water requirements. Since there is not enough water to satisfy all the requests, the programme cannot treat every zone equally. Each zone needs to be considered based on its situation and priority.

Municipal Hospital (Z01) is the most important zone because it deals directly with flood casualties. The hospital needs a dependable minimum amount of water to continue its essential work. It has not waited for any previous cycles, so its priority mainly comes from its critical role rather than its waiting time.

Fire and Emergency Service (Z06) has the smallest request, which is 2,000 litres, and also has the lowest distribution loss of only 1%. It is important for handling emergency situations and is also relatively efficient to supply because very little water is lost during distribution. It has also not waited for any previous cycles.

Central Flood Shelter (Z02) and School Emergency Shelter (Z05) provide temporary accommodation for people who have been displaced by the flood. They need water for the basic needs of the people staying there. Both shelters have already waited for one previous cycle, which gives them some extra priority from a fairness point of view. Their distribution losses are 4% for Z02 and 5% for Z05.

Ward 3 Residential Area (Z03) and Ward 5 Residential Area (Z04) are residential zones with the highest water requests, asking for 4,500 litres and 3,800 litres, respectively. However, they also have the highest distribution losses, with 8% for Z03 and 12% for Z04. Their minimum requirements are also relatively small compared with their total requests. On the other hand, these zones have waited the longest, with Z03 waiting for 2 cycles and Z04 for 3 cycles.

The residential zones create a difficult part of the allocation problem. Although they have waited longer and therefore deserve consideration, they are less critical than the hospital, emergency service, and shelters. Their higher water losses also make them less efficient to supply.

1.4 Summary of conflicting requirements

2. Propos The scenario involves four main conflicts that the proposed solution needs to deal with:

- **Emergency need vs. fairness** — Hospitals and fire services need a minimum amount of water immediately because they are directly involved in emergency response. However, residential areas that have been waiting for several cycles also deserve to be considered. Their waiting time should not be completely ignored.
- **Complete supply vs. wider coverage** — If the programme gives enough water to fully satisfy only a few zones, the other zones may receive nothing. On the other hand, if the available water is divided too widely, important services may not receive enough to meet their minimum requirements. The programme therefore needs to find a balance between satisfying individual zones and providing some water to everyone.
- **Distribution efficiency vs. social need** — Some zones lose more water during distribution than others. For example, the highest loss in this dataset is 12%. Although supplying these zones is less efficient, they still serve people who need water. Therefore, a zone should not be excluded simply because it has a higher distribution loss.
- **Previous waiting vs. new emergency** — A zone that has been waiting for several cycles should receive some consideration because fairness is important. However, waiting time should not be allowed to take priority over a new emergency situation where a zone has an immediate life-safety need.

2 . Proposed Solution

2.1 Two-phase allocation strategy

The programme divides the water allocation process into two phases. Each phase uses different priority factors because the main goal is different at each stage.

- **Phase 1 — Minimum Guarantee:** In the first phase, water is given to the zones according to their priority order until their minimum water requirements are met at the point of delivery, after taking distribution loss into account. The main focus in this phase is the

importance of each service. The order is Hospital > Emergency Service > Emergency Shelter > Residential. This helps make sure that the zones with the most critical life-safety needs are protected first.

- **Phase 2 — Residual Demand:** After the minimum requirements have been dealt with, any water that remains is used to meet the zones' outstanding requests. The zones are again considered according to their priority. However, in this phase, previous waiting cycles have twice the weight they had in Phase 1. This means that a residential zone that has been waiting for several cycles can move higher in the priority order once the minimum requirements of critical services have already been protected.

This two-phase approach helps deal with the main conflicts in the scenario. Criticality is given the most importance at the beginning so that life-safety needs are protected. Waiting time is then given more importance in the second phase, allowing zones that have waited longer to receive fair consideration. At the same time, zones with higher distribution losses are not completely ignored.

2.2 Alternative strategies considered and rejected

Strategy	Description	Reason for rejection
Single weighted-sum priority (one phase only)	Compute one score per zone from all factors and allocate purely in that order until water runs out.	Cannot separately guarantee that every zone reaches its minimum before any zone receives extra beyond its minimum; a single high-scoring zone could exhaust all remaining water on its full request while a lower-scoring zone remains entirely unserved, even below its minimum.
Proportional (equal percentage) split	Give every zone the same fraction of its requested amount.	Ignores the qualitative difference between a hospital's minimum requirement and a residential zone's optional extra litres (see Section 1.3(b)).
Strict type hierarchy, no waiting influence	Always serve Hospital, then Emergency Service, then Shelters, then Residential, in that fixed order, every cycle.	Residential zones could remain permanently under-served across repeated cycles, which the brief explicitly flags as a fairness failure (see Section 1.3(c)).
Waiting cycles as the dominant factor	Rank zones purely by how long they have waited, regardless of type.	Could delay a hospital's minimum supply behind a residential zone purely because the residential zone waited longer, undermining life-safety priority.

The two-phase design was selected because it is the only one of the strategies considered that can guarantee minimums are protected first (addressing the strongest safety concern) while still giving waiting time real, but bounded, influence over the remainder.

2.3 Priority scoring model

Each zone receives a numeric priority score in each phase, computed once per phase and reused for both sorting and allocation:

where typeWeight is 100 for Hospital, 90 for Emergency Service, 70 for Emergency Shelter and 50 for Residential, and waitFactor is 3 in Phase 1 and 6 in Phase 2. Applying this formula to the original dataset gives the following worked scores, which can be cross-checked against the allocation order printed by the programme in Section 5:

Zone ID	Service Zone	Type Weight	Waiting Cycles	Phase-1 Score (+3/cycle)	Phase-2 Score (+6/cycle)
---------	--------------	-------------	----------------	--------------------------	--------------------------

Z01	Municipal Hospital	100	0	100	100
Z02	Central Flood Shelter	70	1	73	76
Z03	Ward 3 Residential Area	50	2	56	62
Z04	Ward 5 Residential Area	50	3	59	68
Z05	School Emergency Shelter	70	1	73	76
Z06	Fire and Emergency Service	90	0	90	90

Sorting the Phase-1 scores in descending order gives Z01 (100), Z06 (90), Z02 = Z05 (73, tied), Z04 (59), Z03 (56) — exactly the order in which the programme protects minimums (see the Case 1 output in Section 5). Sorting the Phase-2 scores gives Z01 (100), Z06 (90), Z02 = Z05 (76, tied), Z04 (68), Z03 (62), matching the residual-demand order reported by the programme.

2.4 Tie-breaking rule

Sometimes, two or more zones may have the same priority score. In that situation, the programme uses three rules in a fixed order to decide which zone should come first.

1. **Waiting time:** The zone that has waited for more previous cycles is given priority. This gives some advantage to zones that have been waiting longer.
2. **Unmet demand:** If the waiting cycles are also the same, the zone with the larger unmet demand is placed first. Unmet demand is calculated as the requested amount minus the minimum requirement. This gives priority to the zone with the greater remaining need.
3. **Zone ID:** If both of the above values are still equal, the zone with the lower Zone ID is placed first. This makes sure that the programme always produces the same result when the input values are identical.

In the original dataset, Z02 and Z05 have the same priority score in both phases. Both are Emergency Shelters and both have 1 previous waiting cycle, so the first rule cannot separate them. The second rule is then checked. Z02 has an unmet demand of $3,500 - 2,500 = 1,000$ L,

while Z05 has $2,600 - 1,600 = 1,000$ L. Since these values are also equal, the third rule is used. Z02 has the lower Zone ID, so it is placed before Z05. This gives Z02 rank 3 and Z05 rank 4, which matches the result produced by the programme.

A second tie is tested in **Section 6.4 (Case 4)**. In that case, the waiting cycles are equal but the unmet demands are different, so Rule (2) is enough to decide the order. This helps confirm that the tie-breaking rules are working correctly at each level.

2.5 Treatment of distribution loss

Distribution loss is deliberately built into the release calculation rather than left as a separate afterthought. To deliver N litres to a zone with loss percentage L , the centre must release:

For example, to deliver the hospital's 3,000 L minimum with its 2% loss, the centre must release $3,000 / 0.98 = 3,061.22$ L, not 3,000 L. This release value

— not the raw requested or minimum figure — is what the allocation logic compares against the water remaining at the centre, and it is what is actually subtracted from the available supply. The full worked release figures for every zone's minimum requirement are shown in Section 5.3.

A deliberate design decision was made not to let loss percentage affect which zones are chosen (i.e. it does not lower a zone's priority score), only how much must be released to serve them. This keeps the priority decision focused on human need rather than distribution economics, while the release calculation still ensures the total water leaving the centre never exceeds the available supply. Section 6.5 (Case 5) demonstrates the effect of increasing one zone's loss percentage: that zone requires substantially more released water to reach the same minimum, which reduces the water available to lower-priority zones — showing that loss still has a real, traceable effect on the outcome even though it does not alter priority ranking directly.

3. Programme Design

3.1 Data organisation

Each service zone is represented by one Zone struct holding its static input fields together with fields computed during allocation. All six zones are held in a single fixed-size array, Zone zones[NUM_ZONES]:

Field	Type	Purpose
code, name, typeLabel	char[]	Zone identifier and display text (e.g. "Z04", "Ward 5 Residential Area").
type	enum ZoneType	Hospital / Emergency Service / Shelter / Residential — used to look up the type weight.
requested, minRequired	double	Input demand figures, in litres.
lossPercent	double	Distribution loss percentage between the centre and this zone.
waitingCycles	int	Number of previous cycles this zone went unresolved.
priority1, priority2	double	Computed Phase-1 and Phase-2 priority scores (Section 2.3).
rank	int	Final allocation-order position (1 = served first).
allocated	double	Litres released from the centre for this zone.
effectiveDelivered	double	Litres actually received after distribution loss.
shortage	double	Unmet requirement at the destination (requested – effective Delivered, floored at 0).
condition	char[]	Final service condition classification (Section 3.2, step 9).

An array of structs was chosen because the number of service zones is small and already known. Using a struct allows all the information related to one zone to be kept together, such as its ID, requested

water, minimum requirement, waiting cycles, and distribution loss. This makes the data easier to manage and reduces the chance of accidentally mixing up values between different zones.

An array also makes it easy to access any zone directly using its index, giving $O(1)$ access time. Instead of making copies of the entire struct array every time the zones need to be viewed in a different order, the programme sorts arrays containing only the integer indices of the zones. The original zone data stays in one place, while the indices can be sorted according to priority, shortage, or zone code.

This approach avoids unnecessary copying of the complete structs and keeps one reliable copy of each zone's information throughout the programme. A more detailed comparison with other possible data structures is provided in Section 4.3.

3.2 Algorithm

The overall algorithm proceeds through ten steps, corresponding directly to the ten numbered functions listed in Section 3.3:

```
1. initialiseZones()           // load the 6 zone records
2. computePhase1Priority()     // type weight + waiting x 3
3. sort index[] by Phase-1 priority (desc, tie-break rule)
4. for each zone in that order:
    release
    =
    minRe
    quired
    / (1 -
    loss/10
    0) give
    =
    min(rel
    ease,
    waterR
    emaini
    ng)
    allocated += give ; waterRemaining -= give
5. computePhase2Priority()     // type weight + waiting x 6
6. sort index[] by Phase-2 priority (desc, tie-break rule)
7. for each zone in
    that order
    (while water
    remains):
    releaseFull =
    requested / (1
    - loss/100)
    stillNeeded =
    releaseFull -
    allocated
    give =
    min(stillNeeded
    ,
    waterRemainin
    g) allocated +=
    give ;
    waterRemainin
    g -= give
```

8. for each zone: $\text{effectiveDelivered} = \text{allocated} \times (1 - \text{loss}/100)$
 $\text{shortage} = \max(0, \text{requested} - \text{effectiveDelivered})$
9. classify each zone's service condition
10. print per-zone summary and aggregate totals

Decision points worth highlighting: step 4 stops giving water to a zone the instant the centre runs dry (give = min(release, waterRemaining)), so no zone can ever push the running total below zero; step 7 skips any zone whose Phase-1 allocation already meets its full request ($\text{stillNeeded} \leq 0$), avoiding a redundant release; and the loop in step 7 is also guarded by "while water remains", so once the centre is empty the remaining zones are left at their Phase-1 allocation without unnecessary iteration.

3.3 Function decomposition

Function	Responsibility
initialiseZones	Loads the six zone records into the array (single point of change for input data).
typeWeight	Returns the criticality weight for a zone's type.
computePhase1Priority / computePhase2Priority	Computes each zone's priority score for the relevant phase; computed once and stored, not recalculated during sorting or allocation.
higherPriority	Comparator implementing the priority ordering plus the three-level tie-break rule.
sortIndicesByPriority	Sorts an index array into priority order (Phase 1 or Phase 2) without moving the underlying zone records.
sortIndicesByShortageDesc	Sorts an index array by remaining shortage, used to report the most under-served zone.
Release Needed For	Converts a destination requirement into the release quantity needed at the centre, given a loss percentage.
Allocate Water	Runs Phase 1 and Phase 2, updates allocated/effective/shortage fields and the final rank.
Classify Conditions	Assigns each zone's final service condition from its effective delivery.
Sort Indices By Code / binary Search By Code	Sorts zones by ID and performs a binary search to retrieve one zone's record by code.
Print Summary / print Allocation Order	Produces the formatted per-zone table and aggregate totals.
main	Reads the available-water quantity (default 13,500 L, optionally overridden by a command-line argument) and drives the workflow.

3.4 Key implementation techniques

Three short extracts from the source (the full listing is given in Appendix A) illustrate the main programming techniques used.

(a) Converting a destination requirement into a release quantity

```
double release Needed For(double needed At Destination, double loss Percent)
{
    return needed At Destination / (1.0 - loss Percent / 100.0);
}
```

A single small function keeps the loss-to-release conversion (Section 2.5) in one place, so it is used identically in both Phase 1 (against minRequired) and Phase 2 (against requested) without duplicating the formula.

(b) The priority comparator and tie-break rule

```
int higherPriority(Zone z[NUM_ZONES], int a, int b, int usePhase2)
{
    double pa = usePhase2
    ? z[a].priority2 :
    z[a].priority1; double
    pb = usePhase2 ?
    z[b].priority2 :
    z[b].priority1; if (pa !=
    pb) return pa > pb;
    if
        (z[a].waitingCycles !=
        z[b].waitingCycles) return
        z[a].waitingCycles >
        z[b].waitingCycles
```

This single comparator is reused for both phases via the usePhase2 flag, so the tie-break logic (Section 2.4) only needs to be written, tested and verified once, rather than duplicated for each phase.

(c) The two-phase allocation loop

```
compute
Phase1Priority(z);
sortIndicesByPriority(z,
idx, 0);
for (int r = 0; r < NUM_ZONE_S; r++)
{ int i = idx[r];
double releaseForMin =
releaseNeededFor(z[i].minRequired, z[i].lossPercent);
double give = (releaseForMin <= available) ?
releaseForMin : available; z[i].allocated += give;
available -= give;
}
```

The `give = (releaseForMin <= available) ? releaseForMin : available` line is the single point in the whole programme responsible for guaranteeing that total allocation never exceeds the available supply: a zone is never given more than the water physically remaining at the centre, regardless of what its calculated release requirement is.

4. Optimisation Considerations

The assignment brief asks that the programme avoid unnecessary processing and redundant storage.

The main goal of the programme was to keep the code efficient while also making it simple enough to understand, read, and check. The programme was not designed to be as short as possible. Instead, the focus was on finding a good balance between efficiency, clarity, and correctness. The following decisions were made for this reason.

- **Sorting using index arrays:** The programme sorts small arrays of integer indices to create the required priority, shortage, and code orders. The actual Zone records are not copied or rearranged. This saves memory and keeps the original zone data organised in one place.
- **Calculating priority once per phase:** Each zone's priority score is calculated once for Phase 1 and once for Phase 2. The scores are stored in the struct before sorting. This means the sorting process only needs to compare the stored values instead of calculating the priority again every time two zones are compared.
- **One allocation pass for each phase:** Each zone is checked once during Phase 1 and at most once during Phase 2. This keeps the allocation process simple and efficient. Once the priority order has been decided, the allocation itself takes linear time because the programme does not repeatedly process the same zones.
- **Using Selection Sort:** Selection Sort was chosen deliberately because there are only **6 zones** in the dataset. With six items, there can be at most 15 comparisons, so an $O(n^2)$ sorting method is more than sufficient. It also makes the sorting and tie-breaking logic easier to understand and check manually. If the number of zones became much larger, a more efficient method such as `qsort()` or another $O(n \log n)$ algorithm would be more appropriate.
- **No hard-coded dataset rules:** The programme uses `NUM_ZONES` for loop limits and compares the actual values stored in each zone rather than checking specific zone IDs. This means the same allocation code can be reused for different test cases without changing the main logic. For example, Cases 1–3 and 7 can use the same code with different available-water values, while Cases 4–6 can use different input data without requiring changes to the allocation algorithm.
- **Calculating loss and shortage at the end:** Effective delivered water and shortage are calculated once in a final pass after all allocations are complete. This avoids unnecessary calculations during the allocation stages and ensures that the final values are based on the zone's total allocation.

4.1 Complexity analysis

Operation	Complexity	Justification
initialiseZones	$O(n)$	One assignment pass over the fixed zone array.
computePhase1Priority / computePhase2Priority	$O(n)$	One arithmetic expression evaluated per zone.
sortIndicesByPriority (per phase)	$O(n^2)$	Selection sort over the index array; acceptable given the small, fixed $n=6$ (Section 4, bullet 4 above).
allocateWater (both phases combined)	$O(n)$	Each zone is visited at most once per phase after the priority order is fixed.
classifyConditions	$O(n)$	One classification decision per zone.
sortIndicesByCode + binarySearchByCode	$O(n^2) + O(\log n)$	One-off selection sort to build the ID-ordered index, after which lookup itself is logarithmic.
Overall programme (one run)	$O(n^2)$	Dominated by the two selection sorts; for $n = 6$ this is at most 30 comparisons in total, which is negligible.

4.2 Memory utilization

The programme uses a single fixed-size array containing six Zone structs, along with a few small integer index arrays of the same size for sorting. Dynamic memory allocation, such as `malloc()` and `free()`, is not used anywhere in the programme.

This makes the memory management much simpler because there is no need to worry about memory leaks or allocation failures. Since the number of zones is already known and limited to six, keeping all the data in a small, fixed block of memory is practical and easy to manage. It also makes the programme more predictable because the amount of memory required does not change while the programme is running.

This approach works well for the current problem because the number of zones is fixed. However, if the programme needed to handle an unknown or very large number of zones, such as data being read from a file with no fixed limit, a dynamically sized array or a linked list would be a better choice. This alternative is discussed in the following section.

4.3 Data structure alternatives considered

Data structure	Advantage for this problem	Reason not selected
Array of structs (selected)	Contiguous memory, $O(1)$ access by index, keeps each zone's fields associated together, simple to sort via an index array.	— (selected)
Parallel arrays (one array per field)	Marginally less struct-definition overhead.	Risks fields becoming misaligned if one array is sorted or updated without the others; the assignment explicitly requires the representation to "maintain the association between related values", which a struct guarantees and parallel arrays do not.
Singly linked list of zone nodes	Would allow the zone count to grow or shrink at runtime without a fixed upper bound.	Adds pointer-management complexity and loses $O(1)$ indexed access, for no benefit here since the zone count is small, known, and fixed for a single distribution cycle.
Hash map keyed by Zone ID	$O(1)$ average-case lookup by ID, faster than the binary search used here for very large n .	Unnecessary overhead (hashing, collision handling) for only six records; binary search over a sorted index array already gives $O(\log n)$ lookup with far simpler code.

5. Implementation and Results

5.1 Compilation

The programme was implemented in ISO C (C11) and compiled without warnings or errors:

```
$ gcc -Wall -Wextra -O2 -o water_allocation water_allocation.c -lm  
(no warnings, no errors)
```

5.2 Case 1 — original dataset, 13,500 L available

The full, unmodified console output for the original dataset is reproduced below.

```
EMERGENCY WATER ALLOCATION SUMMARY (Available: 13500.0 L)
=====
===
ID Zone Requested MinReq Order Allocated Effective Shortage Condition
Z01 Municipal Hospital 4000 3000 1 3477.1 3407.6 592.4 Minimum requirement satisfied
Z02 Central Flood Shelter 3500 2500 3 2604.2 2500.0 1000.0 Minimum requirement satisfied
Z03 Ward 3 Residential Area 4500 2000 6 2173.9 2000.0 2500.0 Minimum requirement satisfied
Z04 Ward 5 Residential Area 3800 1800 5 2045.5 1800.0 2000.0 Minimum requirement satisfied
Z05 School Emergency Shelter 2600 1600 4 1684.2 1600.0 1000.0 Minimum requirement satisfied
Z06 Fire and Emergency Service 2000 1500 2 1515.2 1500.0 500.0 Minimum requirement satisfied

Total water requested : 20400.0 L Total water allocated (released) : 13500.0 L Total effective water delivered :
12807.6 L Total unresolved requirement : 7592.4 L Remaining water at centre : 0.0 L Zones with full
requirement met : 0
Zones with only minimum met : 6 Zones below minimum requirement : 0 Zones completely unserved : 0

Final Allocation Order (by rank):
Z01 - Municipal Hospital
Z06 - Fire and Emergency Service
Z02 - Central Flood Shelter
Z05 - School Emergency Shelter
Z04 - Ward 5 Residential Area
Z03 - Ward 3 Residential Area

Zone with the highest unresolved requirement: Z03 - Ward 3 Residential Area (Shortage: 2500.0 L)

Lookup demonstration - Zone Z04: Allocated=2045.5 L, Effective=1800.0 L, Condition=Minimum requirement
```

5.3 Hand-verification of the Phase-1 release calculations

To confirm the programme's arithmetic independently, the release quantity required for every zone's minimum requirement (Section 2.5) was recomputed by hand:

Zone ID	Minimum (L)	Loss (%)	Release Needed (L)	Calculation
Z01	3,000	2	3,061.22	$3000 / 0.98$
Z02	2,500	4	2,604.17	$2500 / 0.96$
Z03	2,000	8	2,173.91	$2000 / 0.92$
Z04	1,800	12	2,045.45	$1800 / 0.88$
Z05	1,600	5	1,684.21	$1600 / 0.95$
Z06	1,500	1	1,515.15	$1500 / 0.99$

Summing the six release figures gives $3061.22 + 2604.17 + 2173.91 + 2045.45 + 1684.21 + 1515.15 = 13,084.11$ L required to guarantee every zone's minimum. Since only 13,500 L is available, $13,500 - 13,084.11 = 415.89$ L remains for Phase 2. Because Z01 has the highest Phase-2 priority score (Section 2.3, Table), this entire surplus is directed to Z01: $3,061.22 + 415.89 = 3,477.11$ L, which matches the Allocated figure the programme reports for Z01 (3,477.1 L, rounded to one decimal place) exactly. This hand check confirms the programme's Phase-1 and Phase-2 arithmetic is correct for the original dataset.

5.4 Interpretation of results

In Case 1, all six zones manage to receive enough water to meet their minimum requirements. No zone is left below its minimum requirement, but at the same time, none of the zones receives its full requested amount. This is reasonable because the total minimum requirement is 12,400 L, which increases to about 13,084.11 L after taking distribution losses into account. This leaves only a small amount of water available for additional allocation.

The programme uses the full 13,500 L of available water, leaving 0.0 L remaining. Because some water is lost during distribution, the total amount actually delivered to the zones is 12,807.6 L, which is lower than the 13,500 L allocated by the centre.

Among all the zones, Z03 has the largest remaining shortage. This is the Ward 3 Residential Area, which has a relatively large request of 4,500 L but a low Phase-1 priority. As a result, it receives its minimum requirement during the first phase but does not receive much additional water in the second phase. This shows how the priority system affects the final allocation when the available supply is limited.

6. Testing and Validation

Seven conditions were used to validate the programme: the six required situations from the assignment brief, plus one additional, independently designed edge case (Case 7). For every case the same compiled logic was used; only the available-water figure or the relevant input field(s) were changed, confirming that the allocation algorithm itself required no modification between tests.

6.1 Case 1 — Sufficient water for all requested quantities is NOT the case (original scenario)

Test condition: Original dataset, 13,500 L available.

Expected result: Water insufficient for full demand; every zone's minimum protected where possible; total allocated never exceeds 13,500 L.

Available: 13500.0 L -> All 6 zones: Minimum requirement satisfied
Total allocated: 13500.0 L Remaining at centre: 0.0 L
Allocation order: Z01, Z06, Z02, Z05, Z04, Z03
Highest unresolved requirement: Z03 (2500.0 L)
(Full console output reproduced in Section 5.2)

Actual result matches the expected result. Verdict: Pass.

6.2 Case 2 — Available water sufficient for all requested quantities

Test condition: Available water raised to 25,000 L (exceeds total requested 20,400 L).

Expected result: Every zone should receive its complete requested quantity, with water left over at the centre.

EMERGENCY WATER ALLOCATION SUMMARY (Available: 25000.0 L)

ID	Zone	Requested	MinReq	Order	Allocated	Effective	Shortage	Condition
Z01	Municipal Hospital	4000	3000	1	4081.6	4000.0	0.0	Full requirement satisfied
Z02	Central Flood Shelter	3500	2500	3	3645.8	3500.0	0.0	Full requirement satisfied
Z03	Ward 3 Residential Area	4500	2000	6	4891.3	4500.0	0.0	Full requirement satisfied
Z04	Ward 5 Residential Area	3800	1800	5	4318.2	3800.0	0.0	Full requirement satisfied
Z05	School Emergency Shelter	2600	1600	4	2736.8	2600.0	0.0	Full requirement satisfied
Z06	Fire and Emergency Service	2000	1500	2	2020.2	2000.0	0.0	Full requirement satisfied

Total requested: 20400.0 L Total allocated: 21694.0 L Effective delivered: 20400.0 L
Remaining water at centre: 3306.0 L Zones fully satisfied: 6

Actual result matches the expected result. Verdict: Pass.

6.3 Case 3 — Available water insufficient even for the combined minimum requirement

Test condition: Available water reduced to 10,000 L (below the combined minimum requirement of 12,400 L).

Expected result: At least one zone cannot even reach its minimum; the lowest-priority zone(s) show a shortfall or go unserved.

EMERGENCY WATER ALLOCATION SUMMARY (Available: 10000.0 L)

ID	Zone	Requested	MinReq	Order	Allocated	Effective	Shortage	Condition
Z01	Municipal Hospital	4000	3000	1	3061.2	3000.0	1000.0	Minimum requirement satisfied
Z02	Central Flood Shelter	3500	2500	3	2604.2	2500.0	1000.0	Minimum requirement satisfied
Z03	Ward 3 Residential Area	4500	2000	6	0.0	0.0	4500.0	Unserved
Z04	Ward 5 Residential Area	3800	1800	5	1135.2	999.0	2801.0	Below minimum requirement

Z05 School Emergency Shelter 2600 1600 4 1684.2 1600.0 1000.0 Minimum requirement satisfied
 Z06 Fire and Emergency Service 2000 1500 2 1515.2 1500.0 500.0 Minimum requirement satisfied
 Total requested: 20400.0 L Total allocated: 10000.0 L (fully used)
 Zones minimum met: 4 Below minimum: 1 Unserved: 1

Actual result matches the expected result. Verdict: Pass.

6.4 Case 4 — Two or more zones obtain the same priority

Test condition: Z04's waiting cycles changed from 3 to 2, tying its type-and-waiting score with Z03 (13,500 L available).

Expected result: The tie must be resolved by the documented rule (Section 2.4): equal waiting cycles, so decided by unmet demand, not left ambiguous or dependent on array order.

Z03 Phase-1 score = 56, Z04 Phase-1 score = 56 (TIE) Waiting cycles equal (2 = 2) -> compare unmet demand:
 Z03 unmet = 4500-2000 = 2500 L Z04 unmet = 3800-1800 = 2000 L
 Z03 unmet > Z04 unmet -> Z03 ranked ahead of Z04
 Programme result -> Z03 rank 5, Z04 rank 6 (Z03 correctly placed first)

The tie was resolved using level (2) of the rule (unmet demand), exactly as specified in Section 2.4, and not by falling back to Zone ID. Verdict: Pass.

6.5 Case 5 — Effect of increased distribution loss on a residential zone

Test condition: Z04's distribution loss increased from 12% to 40% (13,500 L available).

Expected result: Z04 should require substantially more released water to reach its own minimum, reducing water available to lower-priority zones later in Phase 1.

Z04 minimum release needed: $1800/0.60 = 3000.0$ L (was $1800/0.88 = 2045.5$ L) ID Zone Order Allocated Effective Condition
 Z01 Municipal Hospital 1 3061.2 3000.0 Minimum requirement satisfied Z02 Central Flood Shelter 3 2604.2 2500.0 Minimum requirement satisfied
 Z03 Ward 3 Residential Area 6 1635.2 1504.4 Below minimum requirement <-- was satisfied in Case 1 Z04 Ward 5 Residential Area 5 3000.0 1800.0 Minimum requirement satisfied
 Z05 School Emergency Shelter 4 1684.2 1600.0 Minimum requirement satisfied Z06 Fire and Emergency Service 2 1515.2 1500.0 Minimum requirement satisfied Total effective delivered fell from 12807.6 L (Case 1) to 11904.4 L

Z04's own allocation rose as expected (2045.5 -> 3000.0 L) to still reach exactly 1,800 L delivered. Because Phase 1 is processed in a fixed priority order and Z04 is served before Z03, the extra 954.5 L Z04 now requires is taken from what would otherwise have reached Z03 later in the same phase, which drops from "Minimum requirement satisfied" to "Below minimum requirement". This confirms distribution loss has a real, traceable knock-on effect without changing any zone's priority ranking. Verdict: Pass.

6.6 Case 6 — Effect of increased waiting period on a previously under-served zone

Test condition: Z03's previous waiting cycles increased from 2 to 8 (13,500 L available).

Expected result: Z03's Phase-2 priority score should rise, moving it earlier in the final allocation order.

Z03 Phase-2 score before: $50 + 2 \times 6 = 62$ after: $50 + 8 \times 6 = 98$ Allocation order before: Z01, Z06, Z02, Z05, Z04, Z03 (Z03 rank 6, last) Allocation order after: Z01, Z03, Z06, Z02, Z05, Z04 (Z03 rank 2) Delivered quantities unchanged in this run: the small Phase-2 surplus (415.9 L) still goes to the top-ranked zone (Z01) in both cases.

Z03's rank improved from 6th to 2nd exactly as expected, confirming waiting cycles do influence the Phase-2 order. The delivered litres happened not to change in this particular run, because the available Phase-2 surplus was small enough that only the single top-ranked zone (Z01, unaffected by this change) received any of it in either version — this is noted, not hidden, as an honest observation about this specific numeric case. Verdict: Pass (ordering behaviour correctly changed as designed).

6.7 Case 7 (additional, independently designed) — Independently designed edge case: total supply failure

Test condition: Available water reduced to 0 L.

Expected result: The programme must not divide by zero or crash; every zone should be correctly reported as Unserviced with a shortage equal to its full requested quantity.

```
EMERGENCY WATER ALLOCATION SUMMARY (Available: 0.0 L)
ID Zone
Reque
sted MinReq Order
Allocated Effective
Shortage Condition
Z01 Municipal
Hospital 4000
3000 1 0.0
0.0 4000.0
Unserviced
Z02 Central Flood Shelter 3500 2500 3 0.0 0.0 3500.0 Unserviced
Z03 Ward 3 Residential Area 4500 2000 6 0.0 0.0 4500.0 Unserviced
Z04 Ward 5 Residential Area 3800 1800 5 0.0 0.0 3800.0 Unserviced
Z05 School Emergency Shelter 2600 1600 4 0.0 0.0 2600.0 Unserviced
Z06 Fire and
Emergency Service
2000 1500 2
0.0 0.0
2000.0 Unserviced
Total allocated: 0.0
L Total unresolved:
20400.0 L Zones
unserved: 6
Programme completed normally, no runtime error, no division-by-zero.
```

Actual result matches the expected result. Verdict: Pass.

6.8 Summary of all test cases

Case	Situation tested	Result
1	Original dataset (13,500 L)	Pass
2	Surplus water (25,000 L) — all zones fully satisfied	Pass
3	Below combined minimum (10,000 L)	Pass
4	Equal-priority tie (Z03 vs Z04)	Pass
5	Increased distribution loss (Z04)	Pass
6	Increased waiting cycles (Z03)	Pass
7	Total supply failure — 0 L (additional, student-designed)	Pass

6.9 A noted floating-point boundary effect

During testing, a supplementary check was run with the available quantity set to exactly the theoretical combined minimum-release figure computed in Section 5.3 (13,084.11 L, entered as a rounded literal). One zone was classified “Below minimum requirement” by a fraction of a litre rather than “Minimum requirement satisfied”, because the rounded input literal was marginally smaller than the exact value (13,084.1121...) the classification compares against. This is a genuine floating-point rounding effect of the test input, not a fault in the allocation logic, and is recorded here and under Limitations (Section 7) for completeness.

7. Limitations

There are some limitations in the current programme that should be considered.

Floating-point calculations:

The programme uses floating-point values for water amounts and distribution losses. Because of rounding, comparisons at exact boundary values may sometimes give slightly different results. To reduce this problem, a tolerance of 1×10^{-6} is used when classifying results.

However, if an input value has already been rounded too much, such as entering a release value rounded to two decimal places, the final result can still differ by a small fraction of a litre. In a real system, it would be better to use the original input values without unnecessary rounding.

Input data:

The initial zone information is directly included in the `initialiseZones()` function. The allocation algorithm itself is not designed around any specific result. The test cases were created by changing the input data or available-water value without changing the main allocation logic. However, changing the input currently requires the source code to be edited and compiled again. A better approach would be to read the zone information from a file or from standard input so that different datasets could be tested without recompiling the programme.

Priority weights:

The priority weights used in the programme, such as 100, 90, 70, and 50 for service types and the waiting-cycle factors of 3 and 6, are based on the requirements of this assignment. They provide a consistent way to represent emergency priority and fairness, and they were tested using the seven test cases. However, these values are not based on an official emergency-management standard. If the programme were used in a real flood-response situation, the weights would need to be reviewed and approved by appropriate emergency-management experts.

Single distribution cycle:

The programme only considers one distribution cycle at a time. It does not consider situations where new water becomes available during

the same cycle or where multiple distribution centres are supplying the same zones. It also does not include practical transportation limits, such as the number of vehicles available or how much water can actually be delivered in one trip.

Fixed distribution loss:

The programme assumes that each zone has a fixed distribution-loss percentage throughout the cycle. In a real flood situation, water loss could change depending on road conditions, weather, damaged infrastructure, or the time between deliveries. These changing conditions are not included in the current model.

Waiting-cycle tracking:

The waiting-cycle value is entered as part of the input rather than being updated automatically by the programme. In a real system, the programme should keep track of each zone's allocation from one cycle to the next. For example, if a zone does not receive enough water to meet its requirement, its waiting-cycle count could be increased automatically for the next cycle. This would make the system more suitable for repeated use over a longer emergency period.

8. Recommendations for Future Enhancement

The current programme works well for the given assignment, but several improvements could make it more useful in a real-world situation.

- **File or console-based input:** Instead of keeping all the zone information inside `initialiseZones()`, the programme could read the data from a CSV file or directly from the user. This would allow operators to update the zone information between distribution cycles without changing or recompiling the source code.
- **Automatic waiting-cycle tracking:** The programme could save the results of each distribution cycle and use them in the next cycle. If a zone does not receive enough water to meet its minimum requirement, its waiting-cycle count could be increased automatically. This would make the system more realistic and remove the need to enter waiting-cycle values manually.
- **Configurable priority weights:** The current priority weights could be moved into a configuration file or a separate section of the programme. This would allow the priority given to criticality and waiting time to be changed depending on the needs of a particular municipality or emergency without modifying the main programme.
- **Multi-cycle simulation:** The programme could be extended to run several distribution cycles automatically. The shortage and waiting-cycle information from one cycle could then be carried into the next cycle. This would make it possible to study how the allocation system performs over a longer period instead of looking at only one distribution cycle.

- **Transport-capacity limits:** A maximum amount of water that can be delivered to each zone could be added to the programme. This limit could represent restrictions caused by available vehicles, damaged roads, or pipeline capacity. The allocation process would then consider both the available water and the physical delivery capacity.

9. Conclusion

The two-phase, priority-based approach developed for this assignment provides a clear and consistent way to deal with the water shortage problem. There is no single formula that can be considered the perfect solution, so the programme focuses on balancing the most important factors. First, it protects the minimum water requirements of the most critical services, and then it uses the remaining water to address the outstanding needs of the zones. Waiting time is given more importance in the second phase, which helps make the allocation fairer to zones that have been waiting for longer.

Distribution loss is also included in the water-release calculation. This means the programme considers how much water is actually needed to deliver the required amount to each zone. The total amount released never goes beyond the available supply in any of the seven test cases, including the case where no water is available.

Another important strength of the programme is that the allocation logic is based on the input data rather than fixed zone orders or specific expected results. The same programme was able to handle different situations, including surplus water, severe shortages, shortages below the combined minimum requirements, equal-priority zones, increased distribution loss, increased waiting time, and complete water-supply failure. No changes were needed to the main allocation algorithm for these cases.

The manual calculations in Section 5.3, together with the seven successful test cases in Section 6, provide confidence that the programme is working as intended. Overall, the proposed approach provides a practical way to balance emergency needs, fairness, and efficient use of limited water, while also addressing the main conflicts identified at the beginning of the assignment.

10. References

[1] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

GITHUB LINK : <https://github.com/khaled-zzz/LAB-PROJECT/upload/main>